

Software/Firmware: Librerías GPIO

Funciones básicas para manejo digital — Tarjeta Pinguino (PIC18F2550/4550)

Tarea: Crear funciones básicas para manejo digital **Plataforma:** Tarjeta Pinguino — Microcontrolador PIC18F2550 / PIC18F4550

1. Descripción general

Esta librería implementa funciones básicas de manejo digital de pines de entrada/salida (GPIO) para microcontroladores PIC18F2550/4550 usados en la tarjeta Pinguino. Las funciones siguen una nomenclatura estilo Arduino (`pinMode`, `digitalWrite`, `digitalRead`) para facilitar su uso, pero están escritas en C estándar trabajando directamente sobre los registros del PIC (`TRISx`, `LATx`, `PORTx`), por lo que son compatibles tanto con el IDE/bootloader de Pinguino (SDCC), como con MPLAB.

La librería se compone de tres archivos:

Archivo	Contenido
<code>gpio.h</code>	Declaraciones, constantes y definición de pines
<code>gpio.c</code>	Implementación de las funciones GPIO
<code>main.c</code>	Ejemplo de uso (LED + botón)

2. Funciones implementadas

Función	Descripción
<code>gpio_init()</code>	Configura pines analógicos como digitales (ADCON1, CMCON)
<code>pinMode(pin, modo)</code>	Configura un pin como <code>INPUT</code> o <code>OUTPUT</code>
<code>digitalWrite(pin, valor)</code>	Escribe <code>HIGH</code> o <code>LOW</code> en un pin de salida
<code>digitalRead(pin)</code>	Lee el estado lógico de un pin de entrada
<code>digitalToggle(pin)</code>	Invierte el estado actual de un pin de salida
<code>portMode(puerto, modo)</code>	Configura los 8 bits de un puerto como entrada/salida
Función	Descripción
<code>portWrite(puerto, valor)</code>	Escribe un byte completo a un puerto

```
portRead(puerto)
```

Lee el byte completo de un puerto

3. Mapeo de pines

Los pines se identifican combinando el puerto y el bit mediante la macro `MAKE_PIN(puerto, bit)`, generando constantes como `PIN_A0` `PIN_B5` `PIN_C7`, etc. Esto permite referirse a cualquier pin del PIC18F2550/4550 de forma legible sin necesidad de manipular registros directamente.

Código fuente

3.1 gpio.h

```
/* =====
 * gpio.h
 * Librería básica de manejo digital de pines (GPIO)
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 *
 * Funciones tipo Arduino para configurar y manipular pines
 * digitales de entrada/salida.
 *
 * Autor: (tu nombre)
 * Tarea: Software/Firmware - Librerías GPIO
 * ===== */

#ifndef GPIO_H
#define GPIO_H

#include <pl8cxxx.h> // Definiciones del PIC18F (registros, SFRs)

/* ----- Constantes de modo de pin----- */
#define OUTPUT 0
#define INPUT 1

/* -----Constantes de nivel lógico----- */
#define LOW 0
#define HIGH 1

/* ----- Identificadores de pines -----
 * Numeración tipo "Pinguino": se identifican por
 * puerto y bit. Aquí se mapean los pines digitales
 * más usados en la tarjeta Pinguino 4550 (ajusta según
 * tu placa específica si es necesario).
 *
 * Formato del pin: (PUERTO << 4) | BIT
 * Ejemplo: PIN_B0 = puerto B, bit 0
 *
 */
#define PORT_A 0
#define PORT_B 1
#define PORT_C 2
```

```

#define MAKE_PIN(port, bit)  (((port) << 4) | (bit))

/* Pines del puerto A */
#define PIN_A0 MAKE_PIN(PORT_A, 0)
#define PIN_A1 MAKE_PIN(PORT_A, 1)
#define PIN_A2 MAKE_PIN(PORT_A, 2)
#define PIN_A3 MAKE_PIN(PORT_A, 3)
#define PIN_A5 MAKE_PIN(PORT_A, 5)

/* Pines del puerto B */
#define PIN_B0 MAKE_PIN(PORT_B, 0)
#define PIN_B1 MAKE_PIN(PORT_B, 1)
#define PIN_B2 MAKE_PIN(PORT_B, 2)
#define PIN_B3 MAKE_PIN(PORT_B, 3)
#define PIN_B4 MAKE_PIN(PORT_B, 4)
#define PIN_B5 MAKE_PIN(PORT_B, 5)
#define PIN_B6 MAKE_PIN(PORT_B, 6)

/* =====
 * gpio.h
 * Librería básica de manejo digital de pines (GPIO)
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 *
 * Funciones tipo Arduino para configurar y manipular pines
 * digitales de entrada/salida.
 *
 * Autor: (tu nombre)
 * Tarea: Software/Firmware - Librerías GPIO
 * ===== */

#ifndef GPIO_H
#define GPIO_H
-----

#include <pl8cxxx.h>    // Definiciones del PIC18F (registros, SFRs)

/* ----- Constantes de modo de pin----- */
#define OUTPUT        0
#define INPUT         1

/* ----- Constantes de nivel lógico----- */
#define LOW           0
#define HIGH          1

/* ----- Identificadores de pines -----
 * Numeración tipo "Pinguino": se identifican por
 * puerto y bit. Aquí se mapean los pines digitales
 * más usados en la tarjeta Pinguino 4550 (ajusta según
 * tu placa específica si es necesario).
 *
 * Formato del pin: (PUERTO << 4) | BIT
 * Ejemplo: PIN_B0 = puerto B, bit 0
 *
 * */
#define PORT_A        0
#define PORT_B        1
#define PORT_C        2

```

```

#define MAKE_PIN(port, bit)  (((port) << 4) | (bit))

/* Pines del puerto A */
#define PIN_A0 MAKE_PIN(PORT_A, 0)
#define PIN_A1 MAKE_PIN(PORT_A, 1)
#define PIN_A2 MAKE_PIN(PORT_A, 2)
#define PIN_A3 MAKE_PIN(PORT_A, 3)
#define PIN_A5 MAKE_PIN(PORT_A, 5)

/* Pines del puerto B */
#define PIN_B0 MAKE_PIN(PORT_B, 0) #define PIN_B1 MAKE_PIN(PORT_B, 1)
#define PIN_B2 MAKE_PIN(PORT_B, 2) #define PIN_B3 MAKE_PIN(PORT_B, 3)
#define PIN_B4 MAKE_PIN(PORT_B, 4) #define PIN_B5 MAKE_PIN(PORT_B, 5)

```

```

/* =====
 * gpio.h
 * Librería básica de manejo digital de pines (GPIO)
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 *
 * Funciones tipo Arduino para configurar y manipular pines
 * digitales de entrada/salida.
 *
 * Autor: (tu nombre)
 * Tarea: Software/Firmware - Librerías GPIO
 * ===== */

#ifndef GPIO_H
#define GPIO_H

#include <picxxx.h> // Definiciones del PIC18F (registros, SFRs)

/* ----- Constantes de modo de pin----- */
#define OUTPUT 0
#define INPUT 1

/* ----- Constantes de nivel lógico----- */
#define LOW 0
#define HIGH 1

/* ----- Identificadores de pines -----
 * Numeración tipo "Pinguino": se identifican por
 * puerto y bit. Aquí se mapean los pines digitales
 * más usados en la tarjeta Pinguino 4550 (ajusta según
 * tu placa específica si es necesario).
 *
 * Formato del pin: (PUERTO << 4) | BIT
 * Ejemplo: PIN_B0 = puerto B, bit 0
 *
 */
#define PORT_A 0
#define PORT_B 1
#define PORT_C 2
-----

#define MAKE_PIN(port, bit) (((port) << 4) | (bit))

/* Pines del puerto A */
#define PIN_A0 MAKE_PIN(PORT_A, 0)
#define PIN_A1 MAKE_PIN(PORT_A, 1)
#define PIN_A2 MAKE_PIN(PORT_A, 2)
#define PIN_A3 MAKE_PIN(PORT_A, 3)
#define PIN_A5 MAKE_PIN(PORT_A, 5)

/* Pines del puerto B */
#define PIN_B0 MAKE_PIN(PORT_B, 0)
#define PIN_B1 MAKE_PIN(PORT_B, 1)
#define PIN_B2 MAKE_PIN(PORT_B, 2)
#define PIN_B3 MAKE_PIN(PORT_B, 3)
#define PIN_B4 MAKE_PIN(PORT_B, 4)
#define PIN_B5 MAKE_PIN(PORT_B, 5)
#define PIN_B6 MAKE_PIN(PORT_B, 6)

```

```

/* =====
 * gpio.h
 * Librería básica de manejo digital de pines (GPIO)
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 *
 * Funciones tipo Arduino para configurar y manipular pines
 * digitales de entrada/salida.
 *
 * Autor: (tu nombre)
 * Tarea: Software/Firmware - Librerías GPIO
 * ===== */

#ifndef GPIO_H
#define GPIO_H

#include <pl8cxxx.h> // Definiciones del PIC18F (registros, SFRs)

/* ----- Constantes de modo de pin----- */
#define OUTPUT      0
#define INPUT       1

/* ----- Constantes de nivel lógico----- */
#define LOW         0
#define HIGH        1

/* ----- Identificadores de pines -----
 * Numeración tipo "Pinguino": se identifican por
 * puerto y bit. Aquí se mapean los pines digitales
 * más usados en la tarjeta Pinguino 4550 (ajusta según
 * tu placa específica si es necesario).
 *
 * Formato del pin: (PUERTO << 4) | BIT
 * Ejemplo: PIN_B0 = puerto B, bit 0
 *
 * */
#define _PORT_A_ 0
#define PORT_B 1
#define PORT_C 2

#define MAKE_PIN(port, bit) (((port) << 4) | (bit))

/* Pines del puerto A */
#define PIN_A0 MAKE_PIN(PORT_A, 0)
#define PIN_A1 MAKE_PIN(PORT_A, 1)
#define PIN_A2 MAKE_PIN(PORT_A, 2)
#define PIN_A3 MAKE_PIN(PORT_A, 3)
#define PIN_A5 MAKE_PIN(PORT_A, 5)

/* Pines del puerto B */
#define PIN_B0 MAKE_PIN(PORT_B, 0)
#define PIN_B1 MAKE_PIN(PORT_B, 1)
#define PIN_B2 MAKE_PIN(PORT_B, 2)
#define PIN_B3 MAKE_PIN(PORT_B, 3)
#define PIN_B4 MAKE_PIN(PORT_B, 4)
#define PIN_B5 MAKE_PIN(PORT_B, 5)

```

```
#define PIN_B6 MAKE_PIN(PORT_B, 6)
```

```

/* =====
 * gpio.h
 * Librería básica de manejo digital de pines (GPIO)
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 *
 * Funciones tipo Arduino para configurar y manipular pines
 * digitales de entrada/salida.
 *
 * Autor: (tu nombre)
 * Tarea: Software/Firmware - Librerías GPIO
 * ===== */

#ifndef GPIO_H
#define GPIO_H

#include <pl8cxxx.h> // Definiciones del PIC18F (registros, SFRs)

/* ----- Constantes de modo de pin----- */
#define OUTPUT      0
#define INPUT       1

/* ----- Constantes de nivel lógico----- */
#define LOW         0
#define HIGH        1

/* ----- Identificadores de pines -----
 * Numeración tipo "Pinguino": se identifican por
 * puerto y bit. Aquí se mapean los pines digitales
 * más usados en la tarjeta Pinguino 4550 (ajusta según
 * tu placa específica si es necesario).
 *
 * Formato del pin: (PUERTO << 4) | BIT
 * Ejemplo: PIN_B0 = puerto B, bit 0
 *
 * */
#define PORT_A      0
#define PORT_B      1
#define PORT_C      2
-----
#define MAKE_PIN(port, bit) (((port) << 4) | (bit))

/* Pines del puerto A */
#define PIN_A0 MAKE_PIN(PORT_A, 0)
#define PIN_A1 MAKE_PIN(PORT_A, 1)
#define PIN_A2 MAKE_PIN(PORT_A, 2)
#define PIN_A3 MAKE_PIN(PORT_A, 3)
#define PIN_A5 MAKE_PIN(PORT_A, 5)

/* Pines del puerto B */
#define PIN_B0 MAKE_PIN(PORT_B, 0)
#define PIN_B1 MAKE_PIN(PORT_B, 1)
#define PIN_B2 MAKE_PIN(PORT_B, 2)
#define PIN_B3 MAKE_PIN(PORT_B, 3)
#define PIN_B4 MAKE_PIN(PORT_B, 4)
#define PIN_B5 MAKE_PIN(PORT_B, 5)
#define PIN_B6 MAKE_PIN(PORT_B, 6)

```



```

#define PIN_B7 MAKE_PIN(PORT_B, 7)

/* Pines del puerto C */
#define PIN_C0 MAKE_PIN(PORT_C, 0)
#define PIN_C1 MAKE_PIN(PORT_C, 1)
#define PIN_C2 MAKE_PIN(PORT_C, 2)
#define PIN_C6 MAKE_PIN(PORT_C, 6)
#define PIN_C7 MAKE_PIN(PORT_C, 7)

/* ----- Prototipos de funciones públicas----- */

/* Configura un pin como entrada o salida */
void pinMode(unsigned char pin, unsigned char modo);

/* Escribe un nivel lógico (HIGH/LOW) en un pin de salida */
void digitalWrite(unsigned char pin, unsigned char valor);

/* Lee el nivel lógico de un pin de entrada (regresa HIGH o LOW) */
unsigned char digitalRead(unsigned char pin);

/* Invierte el estado actual de un pin de salida */
void digitalWrite(unsigned char pin);

/* Configura todo un puerto (8 bits) como entrada o salida */
void portMode(unsigned char puerto, unsigned char modo);

/* Escribe un byte completo a un puerto */
void portWrite(unsigned char puerto, unsigned char valor);

/* Lee el byte completo de un puerto */
unsigned char portRead(unsigned char puerto);

/* Inicializa configuración general del sistema GPIO
 * (apaga comparadores y ADC en pines digitales, etc.) */
void gpio_init(void);

#endif /* GPIO_H */

```

3.2 gpio.c

```
/* =====
 * gpio.c
 * Implementación de la librería básica de manejo digital (GPIO)
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 * ===== */

#include "gpio.h"

/* ----- Macros internas de acceso a registros -----
 * Cada puerto del PIC18F tiene 3 registros relevantes:
 *   TRISx -> dirección (1 = entrada, 0 = salida)
 *   LATx  -> escritura (se escribe aquí para evitar
 *                   problemas de lectura-modificación)
 *   PORTx -> lectura del valor físico del pin
 * ----- */

/* Obtiene el número de puerto (parte alta) y bit (parte baja) */
#define GET_PORT(pin)  (((pin) >> 4) & 0x0F)
#define GET_BIT(pin)   ((pin) & 0x0F)

/*
 * gpio_init()
 * Configura el PIC para que los pines analógicos/comparadores
 * funcionen como digitales por defecto. Esto es indispensable
 * en el PIC18F2550/4550 porque varios pines de PORTA y PORTB
 * nacen en modo analógico.
 *
 */
void gpio_init(void)
{
    ADCON1 = 0x0F; // Todos los pines AN0-AN4 como digitales
    CMCN    = 0x07; // Apaga los comparadores analógicos
}

/*
 * pinMode()
 * Configura un pin individual como entrada (INPUT) o
 * salida (OUTPUT), manipulando el bit correspondiente
 * del registro TRISx.
 *
 */
void pinMode(unsigned char pin, unsigned char modo)
{
    unsigned char puerto = GET_PORT(pin);
    unsigned char bit    = GET_BIT(pin);

    switch (puerto)
    {
        case PORT_A:
            if (modo == INPUT)
                TRISA |= (1 << bit);
            else
                TRISA &= ~(1 << bit);
            break;
    }
}
```

```
case PORT_B:  
    if (modo == INPUT)
```

```

/* =====
 * gpio.c
 * Implementación de la librería básica de manejo digital (GPIO)
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 * ===== */

#include "gpio.h"

/* ----- Macros internas de acceso a registros -----
 * Cada puerto del PIC18F tiene 3 registros relevantes:
 *   TRISx -> dirección (1 = entrada, 0 = salida)
 *   LATx  -> escritura (se escribe aquí para evitar
 *                   problemas de lectura-modificación)
 *   PORTx -> lectura del valor físico del pin
 * ----- */

/* Obtiene el número de puerto (parte alta) y bit (parte baja) */
#define GET_PORT(pin)  (((pin) >> 4) & 0x0F)
#define GET_BIT(pin)   ((pin) & 0x0F)

/* -----
 * gpio_init()
 * Configura el PIC para que los pines analógicos/comparadores
 * funcionen como digitales por defecto. Esto es indispensable
 * en el PIC18F2550/4550 porque varios pines de PORTA y PORTB
 * nacen en modo analógico.
 * ----- */
void gpio_init(void)
{
    ADCON1 = 0x0F; // Todos los pines AN0-AN4 como digitales
    CMCON  = 0x07; // Apaga los comparadores analógicos
}

/* -----
 * pinMode()
 * Configura un pin individual como entrada (INPUT) o
 * salida (OUTPUT), manipulando el bit correspondiente
 * del registro TRISx.
 * ----- */
void pinMode(unsigned char pin, unsigned char modo)
{
    unsigned char puerto = GET_PORT(pin);
    unsigned char bit    = GET_BIT(pin);

    switch (puerto)
    {
        case PORT_A:
            if (modo == INPUT)
                TRISA |= (1 << bit);
            else
                TRISA &= ~(1 << bit);
            break;

        case PORT_B:
            if (modo == INPUT)

```

```

        TRISB |= (1 << bit);
    else
        TRISB &= ~(1 << bit);
    break;

case PORT_C:
    if (modo == INPUT)
        TRISC |= (1 << bit);
    else
        TRISC &= ~(1 << bit);
    break;
}
}

/* -----
 * digitalWrite()
 * Coloca un nivel lógico HIGH o LOW en un pin que haya sido
 * previamente configurado como salida. Se usa LATx en vez de
 * PORTx para escribir, ya que es más seguro en PIC18F.
 * ----- */
void digitalWrite(unsigned char pin, unsigned char valor)
{
    unsigned char puerto = GET_PORT(pin);
    unsigned char bit    = GET_BIT(pin);

    switch (puerto)
    {
        case PORT_A:
            if (valor == HIGH)
                LATA |= (1 << bit);
            else
                LATA &= ~(1 << bit);
            break;

        case PORT_B:
            if (valor == HIGH)
                LATB |= (1 << bit);
            else
                LATB &= ~(1 << bit);
            break;

        case PORT_C:
            if (valor == HIGH)
                LATC |= (1 << bit);
            else
                LATC &= ~(1 << bit);
            break;
    }
}

/* -----
 * digitalRead()
 * Lee el estado físico actual de un pin configurado como
 * entrada. Devuelve HIGH (1) o LOW (0).
 * ----- */

```

```

unsigned char digitalRead(unsigned char pin)
{
    unsigned char puerto = GET_PORT(pin);
    unsigned char bit     = GET_BIT(pin);
    unsigned char valor   = 0;

    switch (puerto)
    {
        case PORT_A:
            valor = (PORTA >> bit) & 0x01;
            break;

        case PORT_B:
            valor = (PORTB >> bit) & 0x01;
            break;

        case PORT_C:
            valor = (PORTC >> bit) & 0x01;
            break;
    }

    return valor;
}

/* -----
 * digitalToggle()
 * Invierte el estado actual de un pin de salida.
 * Útil para parpadear un LED sin llevar una variable externa.
 * ----- */
void digitalToggle(unsigned char pin)
{
    if (digitalRead(pin) == HIGH)
        digitalWrite(pin, LOW);
    else
        digitalWrite(pin, HIGH);
}

/* -----
 * portMode()
 * Configura los 8 bits de un puerto completo como entradas
 * o salidas en una sola instrucción.
 * ----- */
void portMode(unsigned char puerto, unsigned char modo)
{
    unsigned char valor = (modo == INPUT) ? 0xFF : 0x00;

    switch (puerto)
    {
        case PORT_A: TRISA = valor; break;
        case PORT_B: TRISB = valor; break;
        case PORT_C: TRISC = valor; break;
    }
}

/* -----

```

```

    * portWrite()
    * Escribe un byte completo a un puerto (los 8 pines a la vez).
    * ----- */
void portWrite(unsigned char puerto, unsigned char valor)
{
    switch (puerto)
    {
        case PORT_A: LATA = valor; break;
        case PORT_B: LATB = valor; break;
        case PORT_C: LATC = valor; break;
    }
}

/* -----
    * portRead()
    * Lee el byte completo de un puerto.
    * ----- */
unsigned char portRead(unsigned char puerto)
{
    unsigned char valor = 0;

    switch (puerto)
    {
        case PORT_A: valor = PORTA; break;
        case PORT_B: valor = PORTB; break;
        case PORT_C: valor = PORTC; break;
    }

    return valor;
}

```

3.3 main.c — Ejemplo de uso

```
/* =====
 * main.c
 * Ejemplo de uso de la librería gpio.h / gpio.c
 * Tarjeta: Pinguino (PIC18F2550 / 4550)
 *
 * Funcionalidad de prueba:
 * - PIN_C0 configurado como salida -> mueve un LED
 * - PIN_B0 configurado como entrada -> lee un botón
 * - Si el botón está presionado, el LED se enciende fijo.
 *   Si no, el LED parpadea.
 * ===== */

#include "gpio.h"

#define LED_PIN    PIN_C0
#define BUTTON_PIN PIN_B0

/* Retardo simple por software (ajusta según tu reloj/oscilador) */
void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    for (i = 0; i < ms; i++)
        for (j = 0; j < 800; j++)
            ; // ciclo vacío
}

void main(void)
{
    gpio_init(); // Inicializa pines analógicos como
digitales

    pinMode(LED_PIN, OUTPUT); // LED como salida
    pinMode(BUTTON_PIN, INPUT); // Botón como entrada

    while (1)
    {
        if (digitalRead(BUTTON_PIN) == HIGH)
        {
            /* Botón presionado: LED encendido fijo */
            digitalWrite(LED_PIN, HIGH);
        }
        else
        {
            /* Botón libre: LED parpadea */
            digitalToggle(LED_PIN);
            delay_ms(300);
        }
    }
}
```


4. Notas de compilación

- El header `p18cxxx.h` incluido en `gpio.h` corresponde al header genérico de dispositivos PIC18F. Si el toolchain utilizado (Pinguino IDE / SDCC / MPLAB) requiere un header específico (por ejemplo `p18f4550.h`), basta con cambiar esa línea de `#include`.
- `gpio_init()` debe llamarse al inicio de `main()`, antes de configurar cualquier pin, ya que deshabilita los módulos analógicos (ADC) y los comparadores que por defecto ocupan varios pines de los puertos A y B.
- Las escrituras se realizan sobre los registros `LATx` en lugar de `PORTx` para evitar el problema clásico de lectura-modificación-escritura (read-modify-write) del PIC18F.